

Continual Learning & Memory Models

COMS 6998

Week #9: March 25, 2026

Paper title: Hopfield Nets is All You Need

Motivation

Core Idea: Intelligence requires associative memory

Retrieve relevant patterns from noisy input.

Hopfield Networks (1982)

Energy Based, associative memory
Stores and retrieves patterns
Convergence Guarantees
Low capacity
Slow retrieval

Transformers - Attention (2017)

Use Attention Mechanisms
Extremely effective in NLP Tasks
Softmax weighted retrieval
Scales well
Strong performance
No memory interpretation

Can we unify memory + attention?

The Key Insight

Modern Hopfield Update

$$\xi_{\text{new}} = \mathbf{X} \cdot \text{softmax}(\beta \mathbf{X}^T \xi)$$

$\xi \rightarrow$ current state (query)

$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] \rightarrow$ stored patterns

$\mathbf{X}^T \xi \rightarrow$ similarity scores

$\text{softmax}(\mathbf{X}^T \xi) \rightarrow$ attention weights

$\beta \rightarrow$ scaling parameter

$\xi_{\text{new}} \rightarrow$ retrieved (weighted) pattern

We introduce a new associative memory model that:

- Retrieves patterns in one step
- Achieves exponential storage capacity
- Has exponentially small retrieval error
- Is fully differentiable

Transformers

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q = Queries, K = Keys, V = Values

Hopfield	Transformer
ξ	Q
$\mathbf{X}$	K, V
update	attention

Transformers perform associative memory retrieval

How does the model work?

Continuous State Space

$$\xi \in \mathbb{R}^d, \quad \mathbf{x}_i \in \mathbb{R}^d$$

Replace binary neurons with continuous vectors

- Enables gradient-based learning
- Supports rich, high-dimensional representations

New Energy Function

$$E(\xi) = -\frac{1}{\beta} \log \left(\sum_{i=1}^N e^{\beta \mathbf{x}_i^T \xi} \right) + \frac{1}{2} \|\xi\|^2$$

Based on log-sum-exp (exponential interactions)

- Amplifies similarity differences
- Suppresses interference
- Creates sharp, well-separated minima

Softmax Update Rule

$$\xi_{\text{new}} = \mathbf{X} \cdot \text{softmax}(\beta \mathbf{X}^T \xi)$$

- Compute similarity $\rightarrow \mathbf{X}^T \xi$
- Apply softmax $\rightarrow$ weights
- Weighted sum $\rightarrow$ output

Continuous $\rightarrow$ Energy $\rightarrow$ Softmax Update
Only the most relevant patterns dominate retrieval

What does the Model actually do?

Global Averaging

- Patterns not well separated
- Softmax $\approx$ uniform weights
- Output = average of all patterns

Diffuse attention

Subset Retrieval

- Some patterns are similar
- Softmax focuses on a subset
- Output = average of similar pattern

Clustered attention

Exact Retrieval

- One pattern dominates
- Softmax is highly peaked
- Output $\approx$ one stored pattern

Sharp attention

Diffuse $\rightarrow$ Focused $\rightarrow$ Sharp
Softmax enables smooth interpolation between memories.

Takeaways

- Attention = Weighting Tokens
- Output = Feature Combination
- Keys/Values = Vectors



- Attention = Associative Memory Retrieval
- Output = Retrieved Pattern
- Keys/Values = Stored Memories

Attention is not just weighting — it is memory retrieval

Theoretical Guarantees

- One-step retrieval
- Exponentially small error
- Converges to stable states

Capacity

- Stores exponentially many patterns
- Scales with dimension

Empirical Impact

- Improves multiple instance learning
- Strong performance on small datasets
- State-of-the-art in several tasks

NEW HOPFIELD LAYERS FOR DEEP LEARNING

- The modern Hopfield network update rule is **(1) differentiable**, so gradients can flow through it during training, and **(2) converges** in one step, so it behaves like a standard layer that's activated once during a forward pass.
- Hopfield network layers can substitute existing layers like pooling layers, permutation equivariant layers, GRU and LSTM layers, and attention layers.
- applications like multiple instance learning, set-based and permutation invariant learning, associative learning via Hopfield Layers

Hopfield energy minimization

$$\xi^{\text{new}} = \underbrace{\mathbf{X}}_{\text{stored patterns}} \text{softmax} \left(\underbrace{\beta}_{\text{inv. temp.}} \underbrace{\mathbf{X}^\top}_{\text{state (query)}} \right)$$

$\longleftrightarrow$

Transformer attention

$$\mathbf{Z} = \text{softmax} \left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V}$$

$$\mathbf{X}^\top \rightarrow \mathbf{K} = \mathbf{Y} \mathbf{W}_K$$

$$\xi \rightarrow \mathbf{Q} = \mathbf{R} \mathbf{W}_Q$$

$$\beta \rightarrow 1/\sqrt{d_k}$$

$$\mathbf{X} \rightarrow \mathbf{V} = \mathbf{Y} \mathbf{W}_K \mathbf{W}_V$$

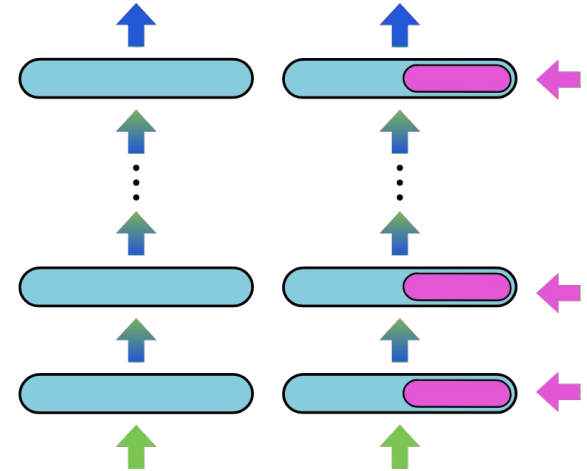
$\mathbf{R} \in \mathbb{R}^{S \times d}$ — raw state patterns (query tokens)

$\mathbf{Y} \in \mathbb{R}^{N \times d}$ — raw stored patterns (memory tokens)

$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ — learned projections into associative space

Self-attention: $\mathbf{Y} = \mathbf{R}$ (every token is both memory & query)

$$\text{Fully expanded: } \mathbf{Z} = \text{softmax} \left(\underbrace{\beta \cdot \mathbf{R} \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{Y}^\top}_{\mathbf{A} \text{ (attn weights)}} \cdot \mathbf{Y} \mathbf{W}_K \mathbf{W}_V \right)$$

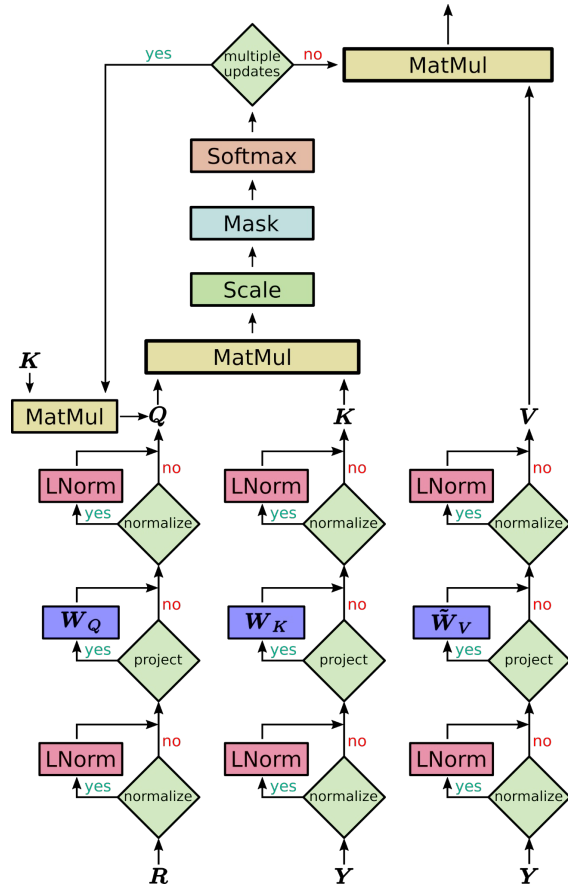


Modern Hopfield Layers

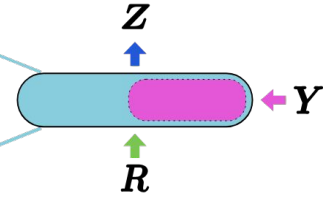
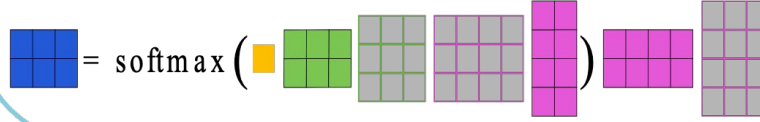
Additional functionalities of the new PyTorch Hopfield layers compared to the transformer (self-)attention layer are:

- **Association of two sets**
 - **Variable β** – that determines the kind of fixed points
 - **Multiple updates** for precise fixed points
 - **Dimension of the associative space** for controlling the storage capacity
 - **Static patterns** for fixed pattern search
 - **Pattern normalization** to control the fixed point dynamics by norm and shift of the patterns
- `Hopfield` for associating and processing two sets. Examples are the transformer attention, which associates keys and queries, and two point sets that have to be compared.
 - `HopfieldPooling` for fixed pattern search, pooling operations, and memories like LSTMs or GRUs. The state (query) pattern is static and can be learned.
 - `HopfieldLayer` for storing fixed patterns or learning internal prototypes. The stored (key) patterns are static and can be learned.

Modern Hopfield Layers - Layer Hopfield



$$\mathbf{Z} = \text{softmax}(\beta \quad \mathbf{R} \quad \mathbf{W}_Q \quad \mathbf{W}_K^T \quad \mathbf{Y}^T) \quad \mathbf{Y} \quad \mathbf{W}_V$$

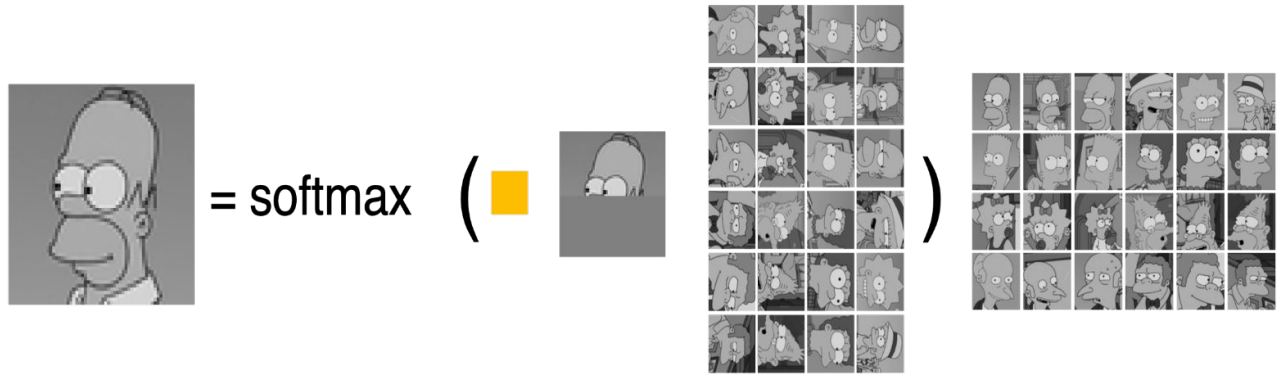
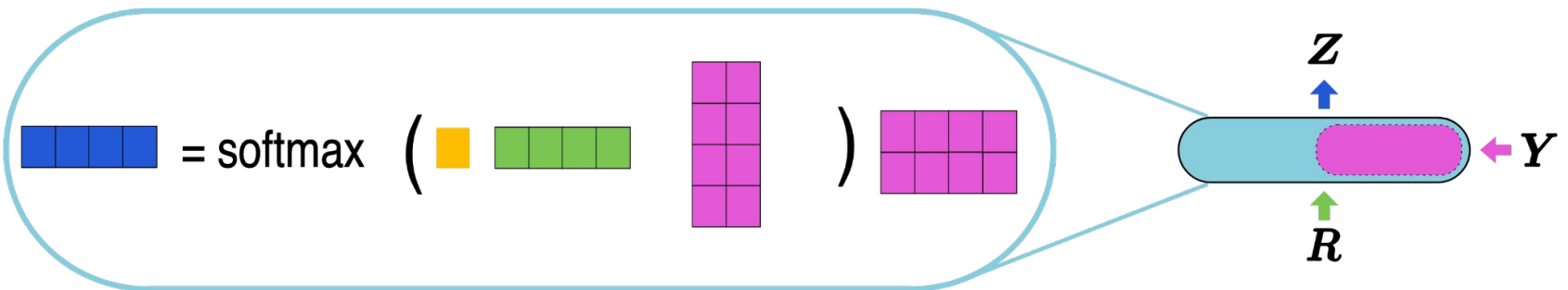


The N raw stored patterns $Y=(y_1, \dots, y_N)^T$ and the S raw state patterns $R=(r_1, \dots, r_S)^T$ are mapped to an associative space via the matrices WK and WQ

We also allow static state and static stored patterns. A static pattern means that it does not depend on the network input, i.e. it is determined by the bias weights and remains constant across different network inputs.

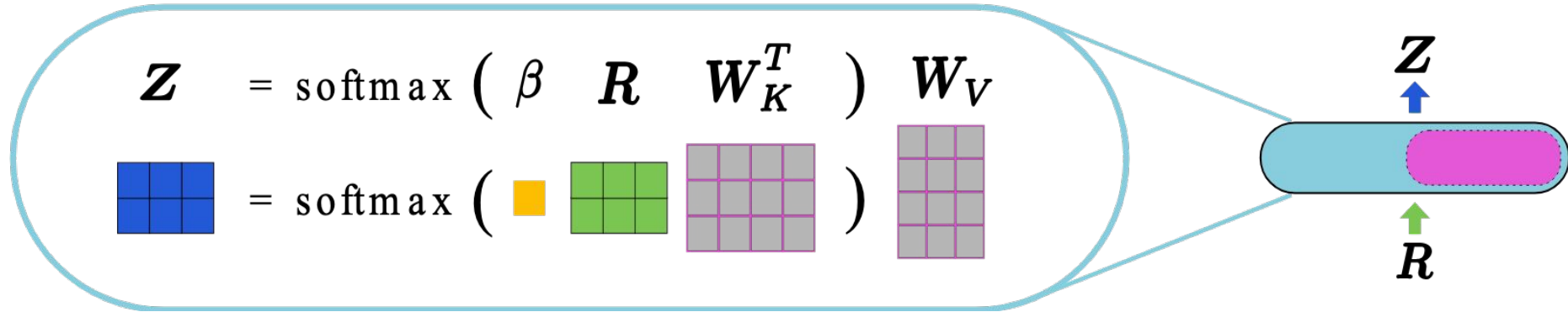
Hopfield layer to solve the pattern retrieval task

$$\mathbf{Z} = \text{softmax} \left(\beta \mathbf{R} \mathbf{Y}^T \right) \mathbf{Y}$$



Hopfield Lookup via HopfieldLayer

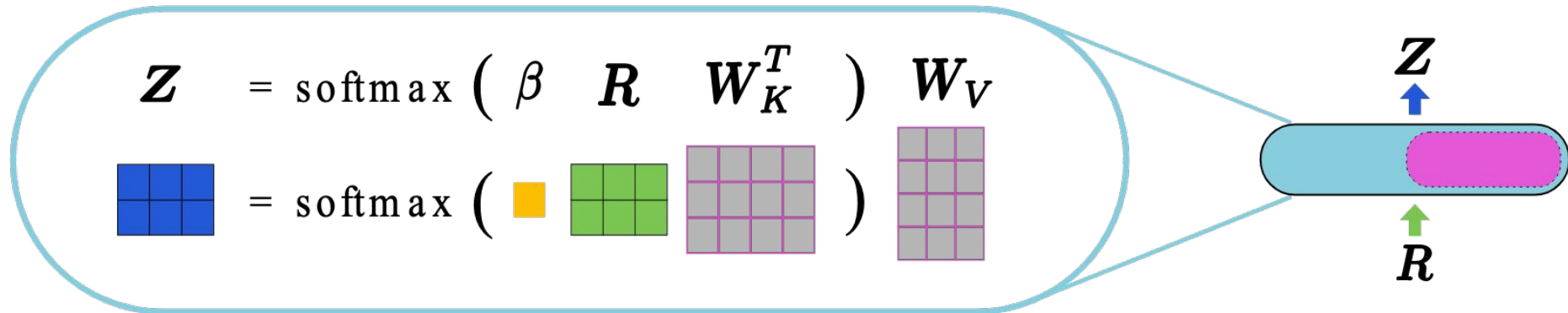
A variant of our Hopfield-based modules is one which employs a **trainable but input independent lookup mechanism**. Internally, one or multiple **stored patterns and pattern projections are trained** (optionally in a non-shared manner), which in turn are used as a lookup mechanism independent of the input data.



HopfieldPooling via HopfieldLayer

We consider the Hopfield layer as a pooling layer if only one static state pattern (query) exists. Then, it is de facto a **pooling over the sequence**. The static state pattern is considered as a **prototype pattern** and consequently learned in the Hopfield pooling layer. Below we give two examples of a Hopfield pooling over the stored patterns $\mathbf{Y}$

Note that the pooling always operates over the token dimension (i.e. the sequence length), and not the token embedding dimension



Modern Hopfield Layers - Experiments

- Attention in Transformers described by Hopfield dynamics .
- **Multiple Instance Learning Datasets**
- Classification on Small UCI Benchmark Datasets
- **Drug Design Benchmark Datasets**

Experiment 1 - DeepRC - Multi-instance learning (MIL)

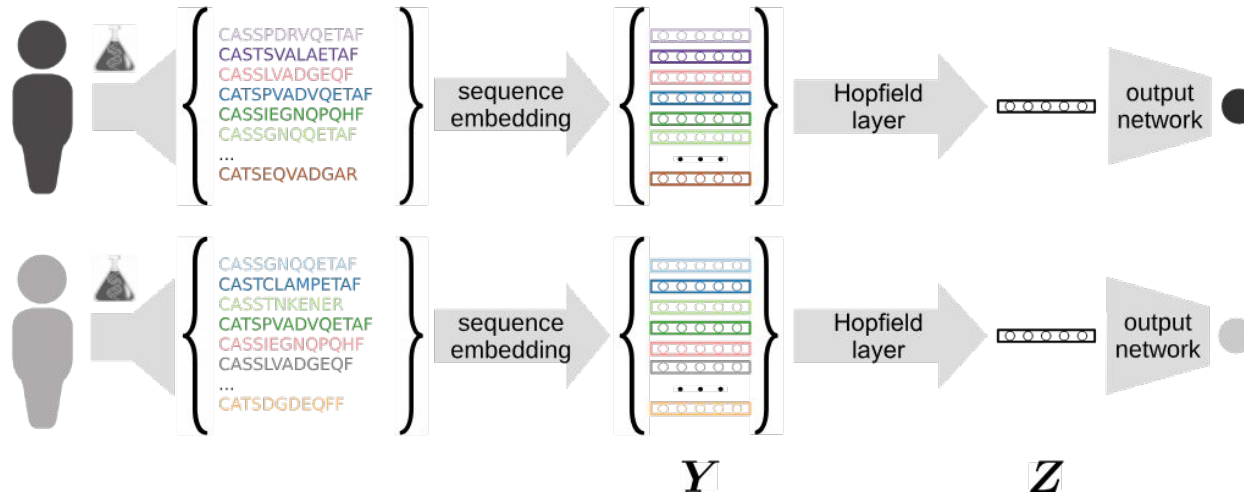
- high storage capacity of Hopfield networks used to solve in computational biology called immune repertoire classification.

An immune repertoire is the total collection of unique antigen receptors—specifically B-cell receptors (BCRs) and T-cell receptors (TCRs)—within an individual's adaptive immune system at a specific time. It acts as a dynamic "immune fingerprint" or historical record of every pathogen a person has ever encountered, as well as their current health status.

- in this MIL problem:
 - Bag = one person's immune repertoire ($\sim 10^5$ – 10^6 sequences after sequencing)
 - Instances = individual receptor sequences (amino acid strings)
 - Bag label = disease status (positive/negative)
 - Key difficulty = maybe only ~ 10 – 100 disease-relevant sequences exist in a sea of hundreds of thousands of irrelevant ones

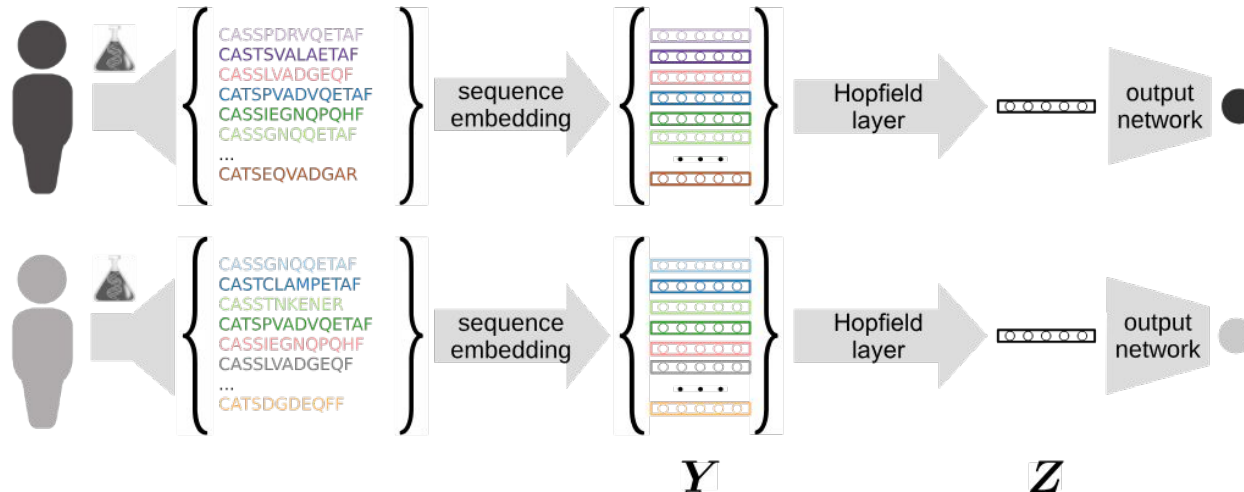
DeepRC - Architecture

- a sequence-embedding neural network to supply a fixed-sized sequence-representation (e.g. 1D-CNN or LSTM),
- a **Hopfield layer part** for sequence-attention, and
- an output neural network and/or fully connected output layer.



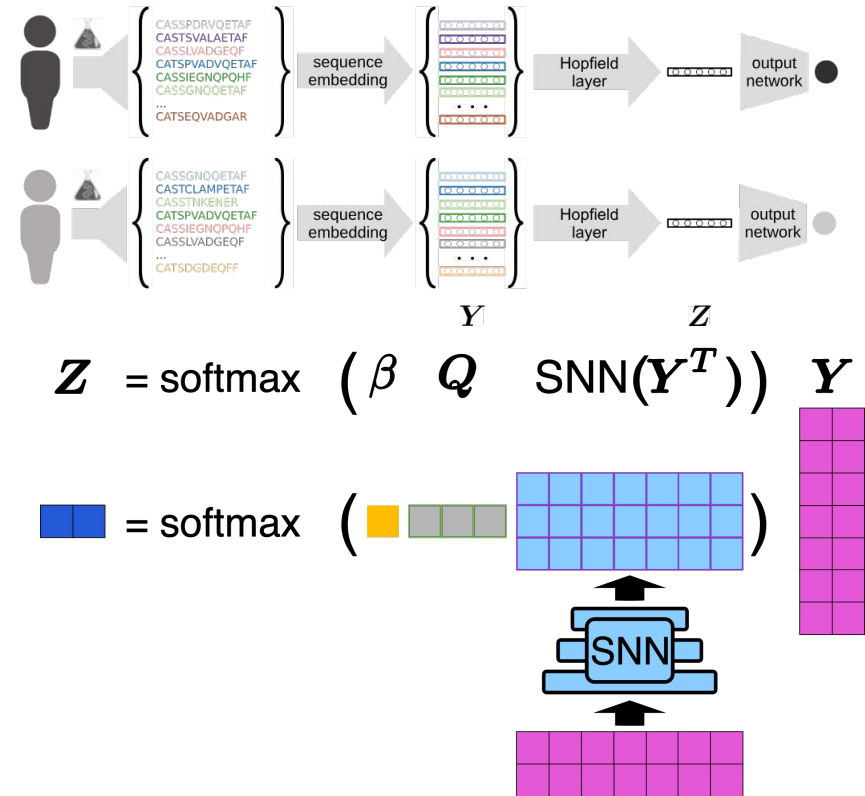
DeepRC - Architecture

- A modern Hopfield network that **extracts information from each repertoire**.
- The stored patterns (keys) are representations of the immune amino acid sequences (instances) that are obtained by an 1D convolutional network with position encoding.
- Each state pattern (query) is static and learned via backpropagation.



DeepRC - Architecture

- the output of the sequence-embedding neural network Y directly acts as values V
- a second neural network, e.g. a self-normalizing neural network (SNN), shares its first layers with the sequence-embedding neural network and outputs the keys K , i.e. the stored patterns, and
- similar to the Hopfield pooling operation, the query vector Q is learned and represents the variable binding sub-sequence we are looking for.
- approx. 300,000 instances per immune repertoire. With over 30 billion sequences in total,



DeepRC - Evaluation Results

Table A.1 reports the average performance in the simulated immunosequencing datasets (last column) and the performance on datasets of the remaining three categories. DeepRC outperforms all competing methods with respect to average AUC. Across categories, the runner-up methods are either the SVM for MIL problems with MinMax kernel or the burden test.

	Real-world	Real-world data with implanted signals				LSTM-generated data				Simulated	
	CMV	OM 1%	OM 0.1%	MM 1%	MM 0.1%	10%	1%	0.5%	0.1%	0.05%	avg.
DeepRC	0.832 ± 0.022	1.00 ± 0.00	0.98 ± 0.01	1.00 ± 0.00	0.94 ± 0.01	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.846 ± 0.223
SVM (MM)	0.825 ± 0.022	1.00 ± 0.00	0.58 ± 0.02	1.00 ± 0.00	0.53 ± 0.02	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.99 ± 0.01	0.827 ± 0.210
SVM (J)	0.546 ± 0.021	0.99 ± 0.00	0.53 ± 0.02	1.00 ± 0.00	0.57 ± 0.02	0.98 ± 0.04	1.00 ± 0.00	1.00 ± 0.00	0.90 ± 0.04	0.77 ± 0.07	0.550 ± 0.080
KNN (MM)	0.679 ± 0.076	0.74 ± 0.24	0.49 ± 0.03	0.67 ± 0.18	0.50 ± 0.02	0.70 ± 0.27	0.72 ± 0.26	0.73 ± 0.26	0.54 ± 0.16	0.52 ± 0.15	0.634 ± 0.129
KNN (J)	0.534 ± 0.039	0.65 ± 0.16	0.48 ± 0.03	0.70 ± 0.20	0.51 ± 0.03	0.70 ± 0.29	0.61 ± 0.24	0.52 ± 0.16	0.55 ± 0.19	0.54 ± 0.19	0.501 ± 0.007
Log. regr.	0.607 ± 0.058	1.00 ± 0.00	0.54 ± 0.04	0.99 ± 0.00	0.51 ± 0.04	1.00 ± 0.00	1.00 ± 0.00	0.93 ± 0.15	0.60 ± 0.19	0.43 ± 0.16	0.826 ± 0.211
Burden test	0.699 ± 0.041	1.00 ± 0.00	0.64 ± 0.05	1.00 ± 0.00	0.89 ± 0.02	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.79 ± 0.28	0.549 ± 0.074
Log. MIL (KMER)	0.582 ± 0.065	0.54 ± 0.07	0.51 ± 0.03	0.99 ± 0.00	0.62 ± 0.15	1.00 ± 0.00	0.72 ± 0.11	0.64 ± 0.14	0.57 ± 0.15	0.53 ± 0.13	0.665 ± 0.224
Log. MIL (TCRβ)	0.515 ± 0.073	0.50 ± 0.03	0.50 ± 0.02	0.99 ± 0.00	0.78 ± 0.03	0.54 ± 0.09	0.57 ± 0.16	0.47 ± 0.09	0.51 ± 0.07	0.50 ± 0.12	0.501 ± 0.016
Known motif b.	–	1.00 ± 0.00	0.70 ± 0.03	0.99 ± 0.00	0.62 ± 0.04	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.890 ± 0.168
Known motif c.	–	0.92 ± 0.00	0.56 ± 0.03	0.65 ± 0.03	0.52 ± 0.03	1.00 ± 0.00	1.00 ± 0.00	0.99 ± 0.01	0.72 ± 0.09	0.63 ± 0.09	0.738 ± 0.202

Table A.1: Results immune repertoire classification across all datasets. Results are given in terms of AUC of the competing methods on all datasets. The reported errors are standard deviations across 5 cross-validation (CV) folds (except for the column “Simulated”). **Real-world CMV:** Average performance over 5 CV folds on the *cytomegalovirus (CMV) dataset Emerson et al. (2017)*. **Real-world data with implanted signals:** Average performance over 5 CV folds for each of the four datasets. A signal was implanted with a frequency (=witness rate) of 1% or 0.1%. Either a single motif (“OM”) or multiple motifs (“MM”) were implanted. **LSTM-generated data:** Average performance over 5 CV folds for each of the 5 datasets. In each dataset, a signal was implanted with a frequency of 10%, 1%, 0.5%, 0.1%, and 0.05%, respectively. **Simulated:** Here we report the mean over 18 simulated datasets with implanted signals and varying difficulties. The error reported is the standard deviation of the AUC values across the 18 datasets.

Methods Evaluated:

- **Known Motif:** Counts how often the known implanted motifs occur in the sequences.
- **Support Vector Machine (SVM):** Uses a fixed mapping from a bag of sequences to the corresponding k-mer counts, with **MinMax** and **Jaccard** kernels.
- **k-Nearest Neighbor (KNN):** Uses k-mer representation; MinMax and Jaccard kernels are transformed into distances.
- **Logistic Regression:** Applied directly on the k-mer representation.
- **Burden Test:** First identifies relevant sequences or k-mers, then computes a burden score per individual.
- **Logistic Multiple Instance Learning (IMIL):** A multiple instance learning approach using logistic regression.
- Note : A k-mer is basically the biological version of an n-gram.

DeepRC - Evaluation Results

Table A.1 reports the average performance in the simulated immunosequencing datasets (last column) and the performance on datasets of the remaining three categories. DeepRC outperforms all competing methods with respect to average AUC. Across categories, the runner-up methods are either the SVM for MIL problems with MinMax kernel or the burden test.

	Real-world	Real-world data with implanted signals				LSTM-generated data					Simulated
	CMV	OM 1%	OM 0.1%	MM 1%	MM 0.1%	10%	1%	0.5%	0.1%	0.05%	avg.
DeepRC	0.832 ± 0.022	1.00 ± 0.00	0.98 ± 0.01	1.00 ± 0.00	0.94 ± 0.01	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.846 ± 0.223
SVM (MM)	0.825 ± 0.022	1.00 ± 0.00	0.58 ± 0.02	1.00 ± 0.00	0.53 ± 0.02	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.99 ± 0.01	0.827 ± 0.210
SVM (J)	0.546 ± 0.021	0.99 ± 0.00	0.53 ± 0.02	1.00 ± 0.00	0.57 ± 0.02	0.98 ± 0.04	1.00 ± 0.00	1.00 ± 0.00	0.90 ± 0.04	0.77 ± 0.07	0.550 ± 0.080
KNN (MM)	0.679 ± 0.076	0.74 ± 0.24	0.49 ± 0.03	0.67 ± 0.18	0.50 ± 0.02	0.70 ± 0.27	0.72 ± 0.26	0.73 ± 0.26	0.54 ± 0.16	0.52 ± 0.15	0.634 ± 0.129
KNN (J)	0.534 ± 0.039	0.65 ± 0.16	0.48 ± 0.03	0.70 ± 0.20	0.51 ± 0.03	0.70 ± 0.29	0.61 ± 0.24	0.52 ± 0.16	0.55 ± 0.19	0.54 ± 0.19	0.501 ± 0.007
Log. regr.	0.607 ± 0.058	1.00 ± 0.00	0.54 ± 0.04	0.99 ± 0.00	0.51 ± 0.04	1.00 ± 0.00	1.00 ± 0.00	0.93 ± 0.15	0.60 ± 0.19	0.43 ± 0.16	0.826 ± 0.211
Burden test	0.699 ± 0.041	1.00 ± 0.00	0.64 ± 0.05	1.00 ± 0.00	0.89 ± 0.02	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.79 ± 0.28	0.549 ± 0.074
Log. MIL (KMER)	0.582 ± 0.065	0.54 ± 0.07	0.51 ± 0.03	0.99 ± 0.00	0.62 ± 0.15	1.00 ± 0.00	0.72 ± 0.11	0.64 ± 0.14	0.57 ± 0.15	0.53 ± 0.13	0.665 ± 0.224
Log. MIL (TCRβ)	0.515 ± 0.073	0.50 ± 0.03	0.50 ± 0.02	0.99 ± 0.00	0.78 ± 0.03	0.54 ± 0.09	0.57 ± 0.16	0.47 ± 0.09	0.51 ± 0.07	0.50 ± 0.12	0.501 ± 0.016
Known motif b.	–	1.00 ± 0.00	0.70 ± 0.03	0.99 ± 0.00	0.62 ± 0.04	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	0.890 ± 0.168
Known motif c.	–	0.92 ± 0.00	0.56 ± 0.03	0.65 ± 0.03	0.52 ± 0.03	1.00 ± 0.00	1.00 ± 0.00	0.99 ± 0.01	0.72 ± 0.09	0.63 ± 0.09	0.738 ± 0.202

Table A.1: Results immune repertoire classification across all datasets. Results are given in terms of AUC of the competing methods on all datasets. The reported errors are standard deviations across 5 cross-validation (CV) folds (except for the column “Simulated”). **Real-world CMV:** Average performance over 5 CV folds on the *cytomegalovirus (CMV) dataset* Emerson et al. (2017). **Real-world data with implanted signals:** Average performance over 5 CV folds for each of the four datasets. A signal was implanted with a frequency (=witness rate) of 1% or 0.1%. Either a single motif (“OM”) or multiple motifs (“MM”) were implanted. **LSTM-generated data:** Average performance over 5 CV folds for each of the 5 datasets. In each dataset, a signal was implanted with a frequency of 10%, 1%, 0.5%, 0.1%, and 0.05%, respectively. **Simulated:** Here we report the mean over 18 simulated datasets with implanted signals and varying difficulties. The error reported is the standard deviation of the AUC values across the 18 datasets.

Performance on the Real-World Dataset:

- **DeepRC:** AUC = 0.832 ± 0.022 (best performing method)
- **SVM with MinMax kernel:** AUC = 0.825 ± 0.022 (second best)
- **Burden Test:** AUC = 0.699 ± 0.041

Drug Design Benchmarks

- The Hopfield layers were tested on 4 challenging classification datasets from MoleculeNet (Wu et al., 2017).
- These datasets cover four important areas in drug discovery:
 - Predicting new anti-virals (HIV)
 - Predicting new protein inhibitors (BACE)
 - Predicting metabolic effects (BBBP)
 - Predicting side effects of chemical compounds (SIDER)
- A Hopfield layer “HopfieldLayer”, in which we used the **training-input as stored pattern Y or key**, the **training-label as pattern-projection $Y \cdot W_V$ or value** and the input as state-pattern R or query.

Table A.5: Results on drug design benchmark datasets. Predictive performance (ROCAUC) on test set as reported by [Jiang et al. \(2020\)](#) for 50 random splits

Model	HIV	BACE	BBBP	SIDER
SVM	0.822 $\pm$ 0.020	0.893 $\pm$ 0.020	0.919 $\pm$ 0.028	0.630 $\pm$ 0.021
XGBoost	0.816 $\pm$ 0.020	0.889 $\pm$ 0.021	0.926 $\pm$ 0.026	0.642 $\pm$ 0.020
RF	0.820 $\pm$ 0.016	0.890 $\pm$ 0.022	0.927 $\pm$ 0.025	0.646 $\pm$ 0.022
GCN	0.834 $\pm$ 0.025	0.898 $\pm$ 0.019	0.903 $\pm$ 0.027	0.634 $\pm$ 0.026
GAT	0.826 $\pm$ 0.030	0.886 $\pm$ 0.023	0.898 $\pm$ 0.033	0.627 $\pm$ 0.024
DNN	0.797 $\pm$ 0.018	0.890 $\pm$ 0.024	0.898 $\pm$ 0.033	0.627 $\pm$ 0.024
MPNN	0.811 $\pm$ 0.031	0.838 $\pm$ 0.027	0.879 $\pm$ 0.037	0.598 $\pm$ 0.031
Attentive FP	0.822 $\pm$ 0.026	0.876 $\pm$ 0.023	0.887 $\pm$ 0.032	0.623 $\pm$ 0.026
Hopfield (ours)	0.815 $\pm$ 0.023	0.902 $\pm$ 0.023	0.910 $\pm$ 0.026	0.672 $\pm$ 0.019

Example : HIV Dataset

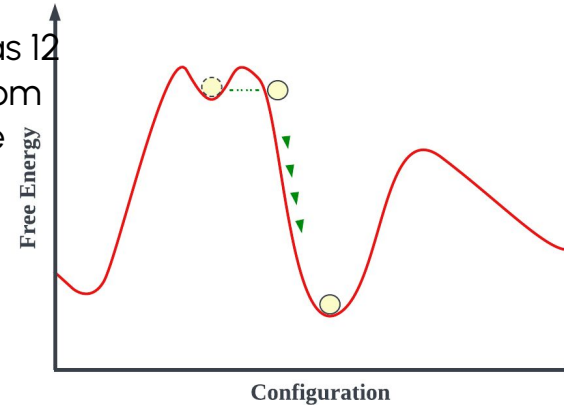
- **Input:** One molecule (SMILES or graph)
- **Output:** Binary label
 - 1 = molecule can inhibit HIV replication (active anti-viral)
 - 0 = inactive
- Task: Predict new anti-viral compounds

Attention in Transformers described by Hopfield dynamics .

How attention heads actually behave in practice — specifically, what kind of fixed points or metastable states they settle into when retrieving patterns from the stored keys/values.

Find the smallest number k such that the top- k weights sum up to at least 0.90 (90% of the total attention mass). for each token

Experimented on : BERT-small setting from ELECTRA (Clark et al., 2020). It has 12 layers, 4 heads and a reduced hidden size, the sequence length is shortened from 512 to 128 tokens and the batch size is reduced from 256 to 128. Additionally, the hidden dimension is reduced from 768 to 256 and the embedding dimension is reduced from 768 to 128



Attention in Transformers described by Hopfield dynamics .



Class (I): Very large metastable state or global fixed point → mostly averaging everything (common in lower layers). (Orange)

Class (II): Large metastable state → averages a large group of tokens. (Pink)

Class (III): Medium metastable state → collects information from a moderate number of relevant tokens (common in final layers). (Green)

Class (IV): Small metastable state → focuses sharply on just a few (or even one) tokens (very common in middle layers 6–8). (Blue)

Observations on the operations classes over training

- Attention in heads of middle layers often change the operation mode to Class IV around 9, 000 to 10, 000 steps , At the same time the second big drop in the loss occurs.
- After the heads switched to **Class IV (small metastable states / sharp attention)**, the **gradient norms became very small and stayed small for the rest of training**. (Frobenius norm of that Jacobian matrix of softmax function.
- a head becomes Class IV, it largely stops learning further. **The attention pattern freezes early and barely changes anymore, even though training continues for another ~1.4 million steps.**

TLDR

Accept, with confidence $\frac{4}{5}$

(+) Strong and thorough theoretical contribution and clear presentation

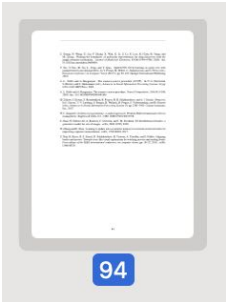
(+) Insightful reinterpretation of attention as associative memory achieved via energy minimization

(-) Practical implications and novelty framing could be clarified

Reviewer: Maria Garmonina

Strengths: Quality, Clarity, Significance for Theory Community

- very clear paper structure (extensive prior work analysis, motivation, contributions, important theorems, benchmarks)
- thorough theoretical & practical evaluation (theory in appendix, 3 dataset classes)
- another way to think about attention (continuous Hopfield associative memory update trained via energy minimization, 3 types of attractor states)



A4	Theorem (Storage Capacity (M=5): Placed Patterns)	47
A5	Theorem (Storage Capacity (Main): Random Patterns)	49
A6	Theorem (Storage Capacity (d computed): Random Patterns)	52
A7	Theorem (Storage Capacity (expected separation): Random Patterns)	55
A8	Theorem (Pattern Retrieval with One Update)	56
A9	Theorem (Exponentially Small Retrieval Error)	57
A10	Theorem (Storage Capacity for Binary Modern Hopfield Nets (Demircigil et al. 2017))	72

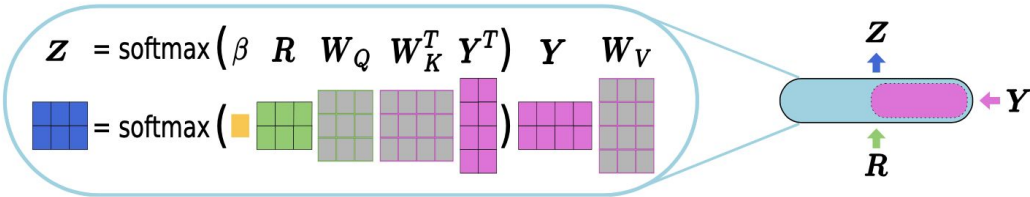


Figure 3: The layer Hopfield allows the association of two sets R (■) and Y (■). It can be integrated into deep networks that propagate sets of vectors. The Hopfield memory is filled with a set from either the input or previous layers. The output is a set of vectors Z (■).

$$E = -\text{lse}(\beta, \mathbf{X}^T \boldsymbol{\xi}) + \frac{1}{2} \boldsymbol{\xi}^T \boldsymbol{\xi} + \beta^{-1} \log N + \frac{1}{2} M^2$$

Weaknesses / Questions:

Practical Implications, Framing

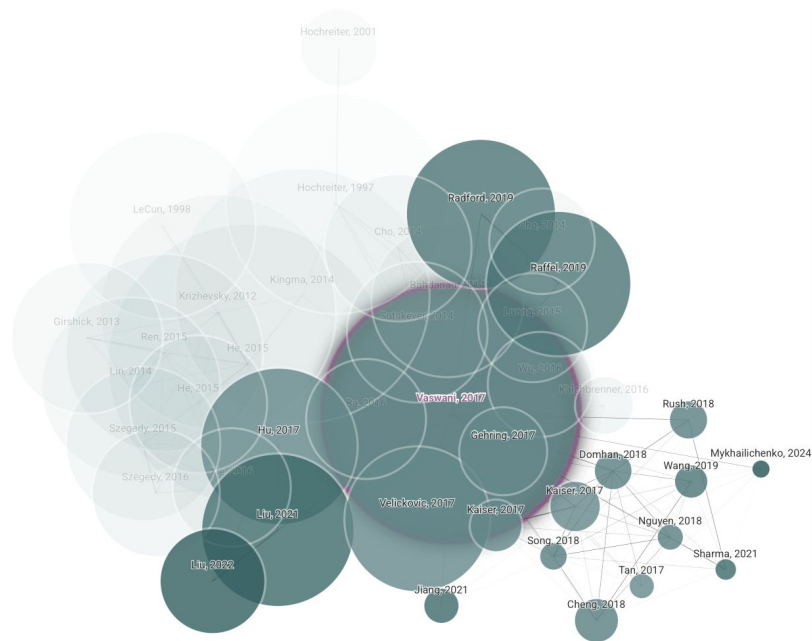
- new paradigm of attention -> so what? implications on architectures, hyperparameters, training strategies, optimizers?
- novelty is mostly conceptual; paper slightly oversells it
- no large-scale tasks in evaluation

Hopfield network layers can substitute existing layers like pooling layers, permutation equivariant layers (Guttenberg et al., 2016; Ravanbakhsh et al., 2016), GRU (Cho et al., 2014) & LSTM (Hochreiter, 1991; Hochreiter & Schmidhuber, 1997) layers, and attention layers (Vaswani et al., 2017a;b; Bahdanau et al., 2014).

We show that our proposed Hopfield layers can be applied successfully to a wide range of tasks.

UCI Benchmark Collection. So far deep learning struggled with small datasets. However, Hopfield networks are promising for handling small datasets, since they can store the training data points or their representations to perform similarity-based, nearest neighbor, or learning vector quantization methods. Therefore, we test the Hopfield layer Hopfield on the small datasets

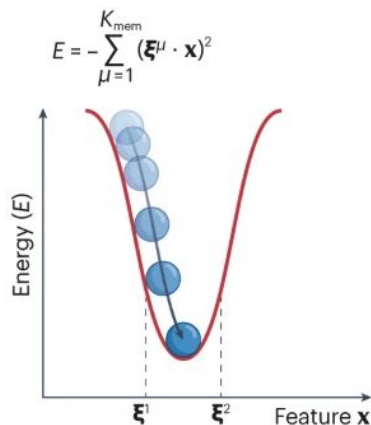
These Hopfield layers enable new ways of deep learning, beyond fully-connected, convolutional, or recurrent networks, and provide pooling, memory, association, and attention mechanisms. Hopfield



Reviewer: Maria Garmonina

Hopfield Networks (1982)

- Associative Memory Network
- Idea was given a set of memories and an input, follow an update rule that would “lead” you to a stored memory



Hopfield Networks Challenges

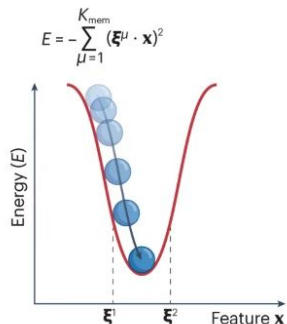
- Smaller Storage Capacity
- Inability to differentiate between similar / highly correlated patterns



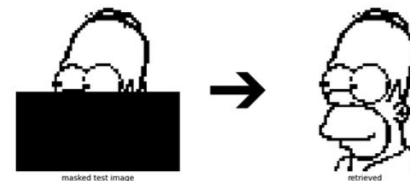
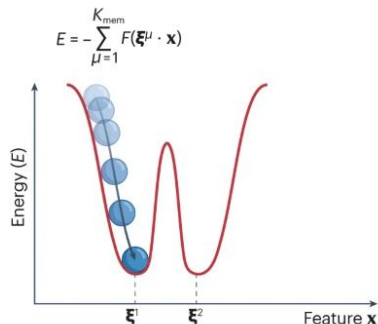
Dense Associative Memories (2017)

- Defined new energy function + update rule
- This allowed for larger storage + faster retrieval

a Traditional Hopfield network



b Dense associative memory



Sources:

Krotov, D. A new frontier for Hopfield networks. Nat Rev Phys 5, 366–367 (2023). <https://doi.org/10.1038/s42254-023-00595-y>
"Hopfield Networks is All You Need (Hopfield Layers)", ML-JKU, <https://ml-jku.github.io/hopfield-layers/>

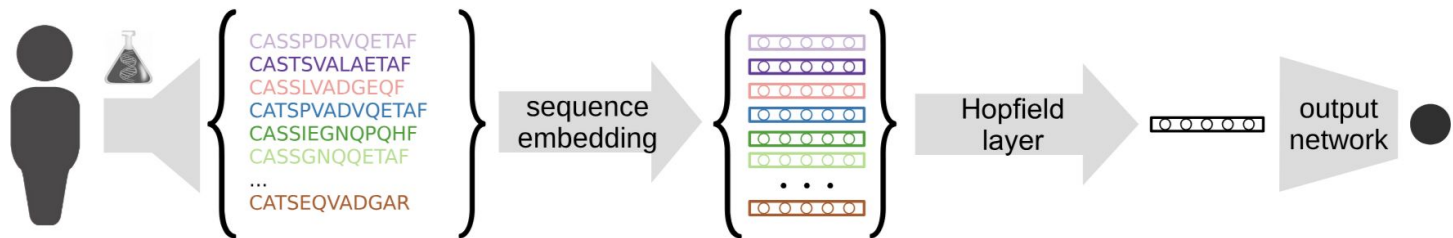
Archeologist: Mateo Juliani

Hopfield Networks is All You Need (2020)

- Generalized Energy Function from prior paper & derive new update rule
- Now continuous + differentiable - could be used in a neural network

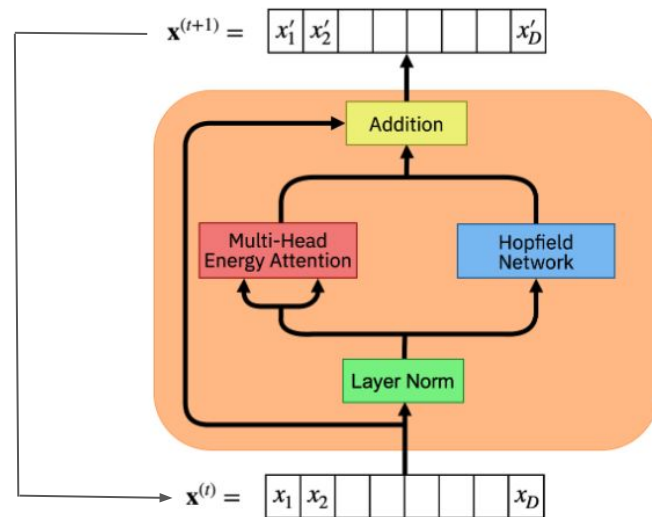
DeepRC (2020)

- Humans have a large amount of immune repertoire receptors (10^7 - 10^8)
- Can also observe whether a human is immune or not to a disease
- Challenge is finding which receptors are associated with immunity
- Hopfield Layers helped reach SoTA results.



Energy Transformers (2023)

- Proposed new energy transformer block + recurrent block architecture
- Achieved SoTA results on graph anomaly detection + classification tasks

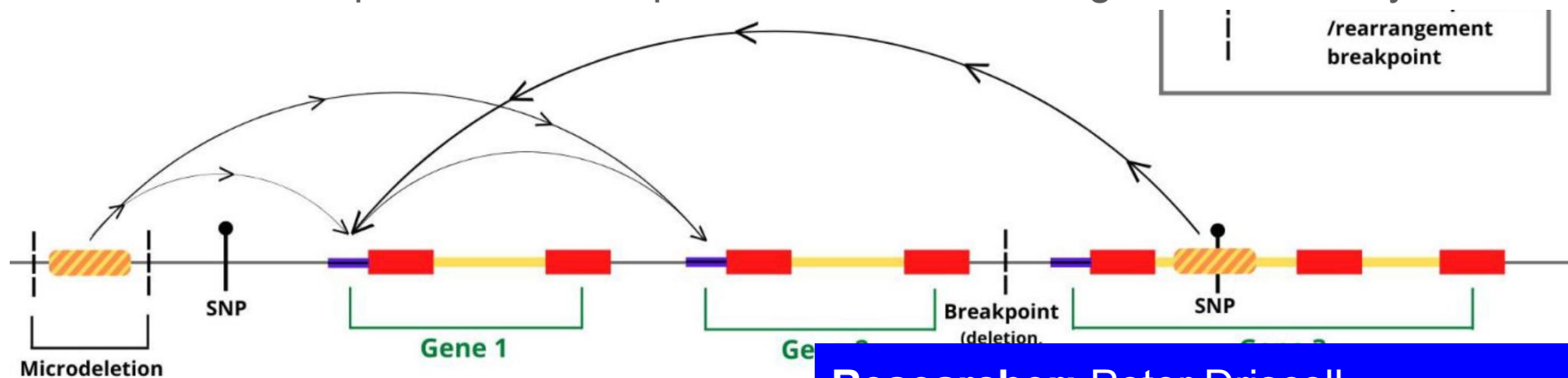


Conclusion

- Provided theoretical grounding behind attention mechanism
- Gained adoption / SoTA results in some fields
- However did not get broad adoption across the ML field

Hopfield Memory for Genomic Perturbation Modeling

- The Hopfield paper reframes attention as associative memory retrieval
- What if we could leverage this idea in genomics to compress far flung regulatory relationships into memory prototypes?
- Use perturbation identity and cell state as the lookup key for retrieval
- Goal: Predict perturbation response from nearest neighbor in memory



Researcher: Peter Driscoll

Long DNA as memory with Perturbation as Query

- Encode long genomic context into candidate regulatory memories by generating embeddings for semantically relevant prototypes using raw sequence data, i.e., DNA_sequence {promoters, enhancers, motifs} -> z_embed
- Query memory with current perturbation and cell state

Not Another Sequence Model

- Existing sequence models encode DNA into a perturbation representation
- This architecture proposes to form the query from the perturbation and use it to retrieve a softmax over associative memories
 - DNA is embedded in the memory bank and the model prediction depends on what was recalled
- Yields stronger inductive bias:
 - encodes broad regulatory context in semantic identity for diffuse entities
 - also enables sharp retrieval for distinct cis-regulatory effects
- Could yield more flexible, generalized prediction of unseen perturbations by endowing shared regulatory patterns in prototype identity

Researcher: Peter Driscoll

Code and Data (Score: 7/10)

Strengths:

- Code is released as an installable pip package (hopfield-layers v1.0.2) with minimal dependencies (torch>=1.5, numpy>=1.20).
- The core library is compact and follows PyTorch conventions closely, since it's explicitly adapted from PyTorch v1.6.0's MultiheadAttention.
- Synthetic datasets (BitPatternSet, LatchSequenceSet) are generated in-code, making the bit-pattern and latch-sequence demos fully self-contained.
- The MNIST-bags example auto-downloads MNIST via torchvision, so no manual data procurement is needed.
- Source code could be found at <https://github.com/ml-jku/hopfield-layers>

Weakness:

- No bundled raw datasets. All data is either synthetic (generated in code) or downloaded at runtime (MNIST).
- The paper's main experiments on immune repertoire classification use proprietary/external datasets not included here. This is a significant gap for full replication.

Bugs:

- Incorrect attribute reference: `Hopfield.normalize_hopfield_space` uses non-existent `self.hopfield` (should be `self.association_core`), leading to `AttributeError` if invoked (`init.py:355–360`)
- Misnamed output layer: `HopfieldDecoderLayer.output_size` references `self.linear_output_self` instead of `self.linear_output`, causing `AttributeError` when accessed (`transformer.py:212`)
- Missing import: `functional.py` calls `warnings.warn` without importing `warnings`, resulting in `NameError` on deprecated byte-tensor code paths (lines 310–311, 348–349)
- Undefined variable: `BitPatternSet.initial_seed` returns `self.__seed`, which is not defined (should use `__seed_signals / __seed_data`), causing failure if called (`auxiliary/data.py:155`)

Hacker: Binyan Sun

Documentation (Score: 9/10)

Strengths:

- The README.md is well-structured: it explains the theoretical motivation, lists authors/affiliations, provides installation instructions, minimal usage snippets, and links to the paper and blog post.
- Each of the 3 example notebooks has output results, so users can see expected results without running anything.
- The mnist_bags example has its own README.md explaining how to integrate with external AttentionDeepMIL code.
- Docstrings on every class and method are thorough, with parameter descriptions.

Weaknesses:

- No API reference site (no Sphinx, MkDocs, or similar). Users must read source code for advanced usage.
- No CONTRIBUTING guide, no changelog, no CI configuration.
- The HopfieldEncoderLayer / HopfieldDecoderLayer (transformer module) are not mentioned in the main README at all — users would have to discover them by browsing the source.
- The mapping between paper equations and code variables is not explicitly documented anywhere. For instance, the relationship between scaling (β), update_steps_max (number of Hopfield retrieval iterations), and the paper's theoretical framework requires the reader to reverse-engineer from code.

Model and Training (model: 8/10, training: 5/10)

Models are fully open. The three main modules are:

Module	Role	Paper Mapping
Hopfield	General Hopfield association ($\approx$ attention layer)	Core update rule (Eq. 4)
HopfieldPooling	Trainable state pattern + Hopfield ($\approx$ pooling)	Pooling via learned query
HopfieldLayer	Trainable stored patterns + Hopfield ($\approx$ lookup)	Fixed memory store

Training:

This is the weakest dimension.

- No standalone training scripts (train.py, CLI, config files). All training is inside Jupyter notebooks.
- There are 3 example notebooks:
 - Bit Pattern demo — binary MIL classification
 - Latch Sequence demo — long-term dependency learning
 - MNIST Bags — MIL on MNIST digit bags
- The paper's main experiments (immune repertoire classification, BERT analysis, large-scale pattern storage) are not reproducible from this repo alone — the necessary datasets, hyperparameters, and training infrastructure are absent.
- No hyperparameter configs, no experiment tracking, no logging infrastructure.

Hacker: Binyan Sun

Extensibility (Score: 8/10)

Strengths:

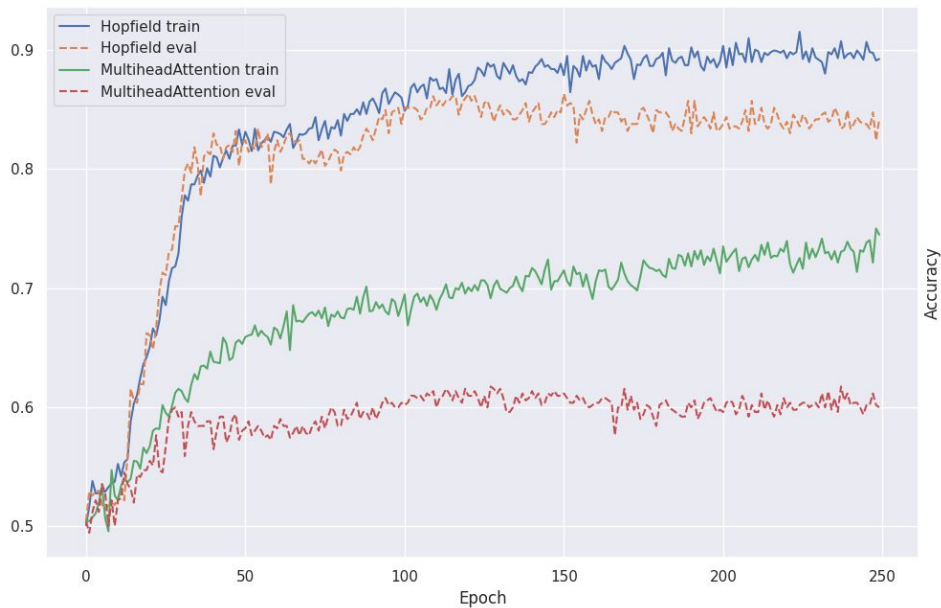
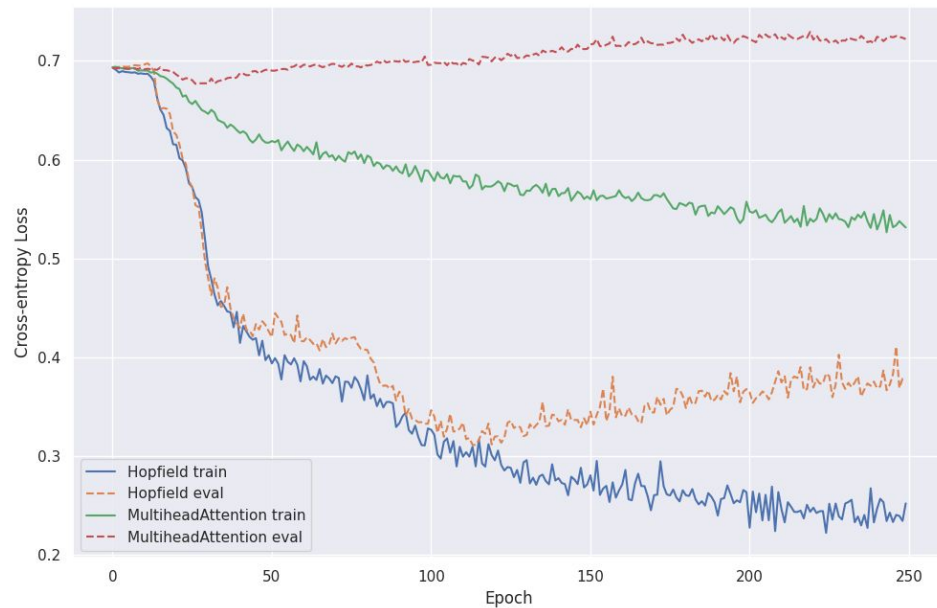
- Modular design (Hopfield $\rightarrow$ HopfieldCore $\rightarrow$ hopfield_core_forward) enables flexible customization of activation, scaling (β), retrieval iterations, and projection patterns
- Supports interchangeable softmax alternatives and per-head scaling for fine-grained control of attention dynamics
- Flexible input interface (batch_first, tensor or tuple inputs) aligns with PyTorch conventions
- Seamless integration: HopfieldEncoderLayer / HopfieldDecoderLayer can be directly used in standard Transformer stacks

Challenges for Extension:

- Monolithic core implementation (~454 lines in functional.py) tightly couples projection, normalization, masking, and iterative updates, making modifications difficult
- Lack of unit tests prevents reliable validation and regression checking after changes
- Outdated PyTorch dependencies (1.6-era APIs) may cause compatibility issues or require adaptation for PyTorch 2.x

Extended Experiment

Hopfield vs. MultiheadAttention on Bit Pattern Set



Source Code:

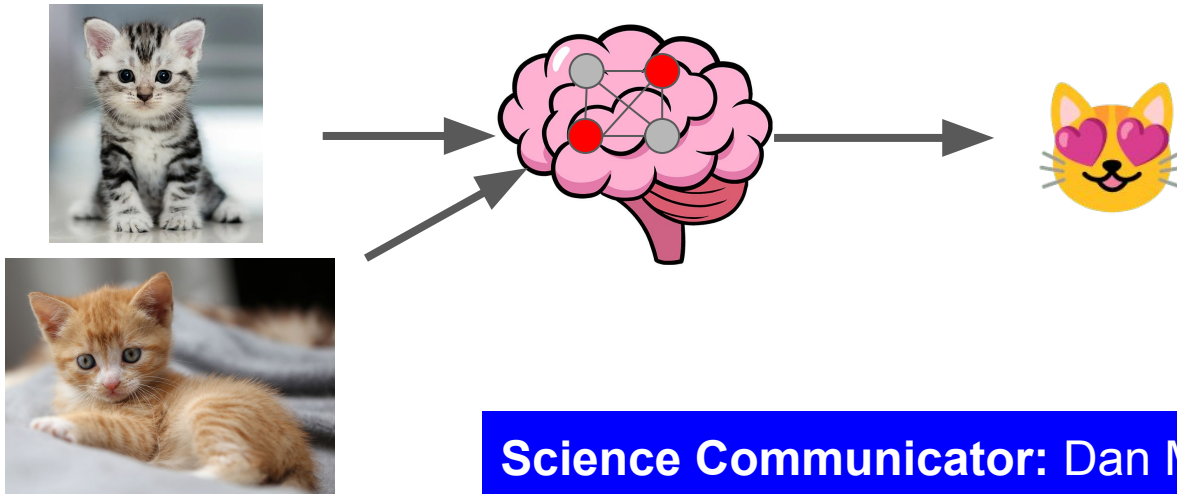
<https://github.com/bob-021206/hopfield-layers.git>

Hacker: Binyan Sun

Science Communicator: Hopfield Networks is All You Need

Associative memory: the brain is incredibly good at filling in missing information or adjusting for noise, and it stores memories in an associative manner where related inputs trigger consistent responses. This is achieved by densely connected neurons activating in stable patterns.

Hebbian Learning: How we believe our brains learn concepts. "Neurons that fire together wire together". Repeated co-stimulation strengthens connections.



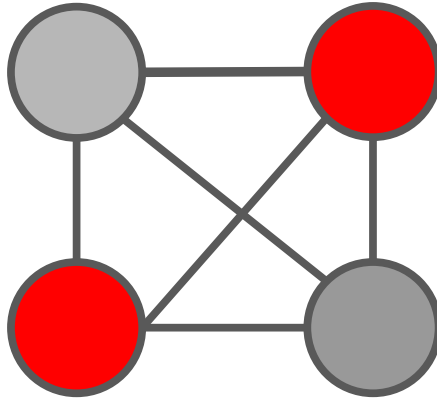
Science Communicator: Dan Meyer (djm2261)

Science Communicator: Hopfield Networks is All You Need

Hopfield Networks: Invented by John Hopfield in 1982, Hopfield Networks are a form of artificial neural network that models associative memory and can be used to recover previously seen patterns from partial or noisy inputs. They have beautiful theoretical connections across many disciplines and Hopfield won a Nobel Prize for developing them in 2024.

Drawbacks of Hopfield Networks:

- Binary valued neurons
- Limited scalability
- Discrete (can't be learned with gradient descent)

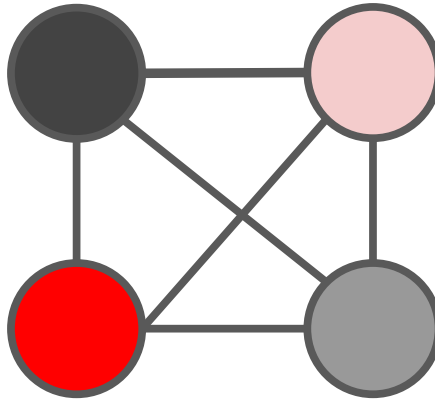


Science Communicator: Dan Meyer (djm2261)

Finally, Hopfield Networks for Deep Learning!

Modern Hopfield Networks introduced in "Hopfield Networks is All You Need" unlock the potential of Hopfield Networks for use in deep learning, by allowing neuron values to be continuous instead of binary (so they can be learned by gradient descent), and also exponentially increase the storage capacity so that they can scale to recall large numbers of memories.

Previously n neurons would be able to store n inputs without errors.
Now, n neurons can store $2^{n/2}$ inputs without errors!



Available today in many shapes and sizes, perfectly suited for your deep learning needs!

<https://github.com/ml-jku/hopfield-layers>

Science Communicator: Dan Meyer (djm2261)

1. We think of the memory capacity of a transformer's context window as the context length. But if you think of self-attention as implementing a Hopfield update, then you get catastrophic forgetting at some ratio of stored patterns per unit. Would you ever hit this limit in a transformer if you just keep increasing the context window or key/value dimension?
2. A self-attention head operates on that layer's keys/values, whereas a Hopfield layer could have in memory patterns from that layer or previous layers/timesteps. Could you meta-learn what information is stored in the hopfield layer's memory?

1. I feel like there is a slight violation in assumptions with Thm 3. (assuming random patterns on a sphere) and the experiments with Immune Repertoire Classification / Drug Design. I would assume that those data would be highly structured and non-random (esp w.r.t some kind of biological order and correlation signals). Wouldn't this non-random structure affect the effective storage capacity + overall performance?
2. I wonder what the Hopfield interpretation would be for a feedforward layer (between attention layers). Since attention layers are considered to be doing energy minimization and memory retrieval, what do feedforward layers act as (e.g. changing the landscapes between attn layers)?

The paper shows high theoretical capacity, but I'm wondering how retrieval behaves in practice when many stored patterns are very similar? Does the softmax still focus on a single pattern, or can the output become less precise because it spreads across multiple similar ones?

The paper states that metastable states enable the network to average over a subset of patterns. Can these metastable states serve as broad concepts instead of specific memories?

Questions/Comments: Emily Bejerano

Questions/Comments: Akshat Bhandari

Questions / Comments: Suhas